

Clustering and Synthesis in Large-Scale Deliberation

A verified-embedding architecture with per-user-deduplicated evaluation, aligned to the deployed Freedi implementation

Tal Yaron

Claude (Anthropic)

2026-07-09

Abstract

Open-ended deliberation produces hundreds to thousands of proposals per question, creating two distinct computational problems: *fragmentation* (the same idea restated in many words, splitting votes across paraphrases) and *organization* (the surviving distinct ideas must be grouped so participants can navigate them). Standard text-embedding clustering fails on both, because modern embeddings encode topical proximity, not propositional content — “raise taxes” and “lower taxes” embed at cosine above 0.92. We describe the clustering-and-synthesis architecture as it is actually deployed in the Freedi platform. The unifying design principle is that embedding cosine similarity is a *candidate generator*, *never a decision rule*: every merge is verified either by community evaluation data or by a separate LLM semantic-equivalence judgment. We give the live single-option pipeline, the bulk admin pipeline (ANN-cosine candidate edges plus a medoid-anchored two-tier judge), the shared per-user-deduplicated evaluation aggregation, the operational-hygiene layer that makes synthesis identifiable/reversible/re-runnable, and an honest account of what is deployed versus what remains implemented-but-unwired. This revision reconciles an earlier draft with the source code: the bulk pipeline uses ANN-cosine + connected components (not UMAP→DBSCAN), hybrid text-rating clustering exists only as unwired primitives, and the numeric thresholds are restated to match the code.

Contents

1. Introduction	2
1.1 The two problems	2
1.2 Why pure embedding similarity is insufficient for either problem	3
1.3 The architecture in one paragraph	3
2. Embedding Infrastructure	4
2.1 Choice of embedding model	4
2.2 Context-aware generation	4
2.3 Storage and retrieval	4
3. Idea Synthesis	4
3.1 Notation and cluster kinds	5
3.2 The live single-option pipeline	5
3.3 The bulk admin pipeline	7
3.4 The four-way verdict and verdict cache	8
3.5 Scheduled consolidation and admin entry points	9

3.6 Member hiding happens only on the execute/integration path	9
4. Hybrid Text–Rating Clustering (designed; primitives implemented; not wired)	10
4.1 The design	10
4.2 What actually runs	10
5. Topic Clustering (deployed)	11
6. Shared Infrastructure	11
6.1 The merge primitive — <code>performIntegration</code>	11
6.2 Per-user-deduplicated evaluation aggregation	12
6.3 Keeping the live cluster aggregate current, and clean-then-rebuild	12
7. Production Hardening and the UMAP→DBSCAN Correction	12
7.1 The incident	13
7.2 Operational hygiene	13
8. Verification, Benchmarks, and Empirical Reproduction	14
8.1 Testing surface	14
8.2 The benchmark harness and its lesson	14
8.3 Recommended regression surface	14
9. Scaling Analysis	15
9.1 Live pipeline (per option write)	15
9.2 Bulk pipeline (per admin run)	15
9.3 The membership-idempotence fix	15
10. Limitations and Open Questions	16
11. Conclusion	16
Appendix A — Reconciliation with the source code	17
References	18

1. Introduction

1.1 The two problems

In open deliberation, participants are not constrained to a fixed list of options — they generate proposals freely. Two regularities follow.

Repetition is structurally inevitable. Two participants who share a viewpoint and never communicated will reliably produce semantically identical proposals in different words. A thousand participants on a single question can produce dozens of proposals expressing one underlying idea. Each variant captures only a fraction of the votes the unified idea would attract:

$$\text{vote}(\text{idea}) = \sum_{v \in \text{variants}(\text{idea})} \text{vote}(v).$$

Comparative metrics — ranking, top- k selection, consensus scoring — are distorted by the arbitrary partition of one idea into N variants.

Volume exceeds direct comparison. Even after removing paraphrases, the surviving distinct ideas still number in the hundreds. Participants and facilitators cannot meaningfully rank such a list directly. They need an organizing structure: thematic groups within which proposals share a subject, between which they differ.

These are different problems with different remedies. *Synthesis* collapses paraphrases of one idea into one canonical proposal. *Clustering* organizes distinct ideas into navigable groups. Both are needed; neither substitutes for the other.

1.2 Why pure embedding similarity is insufficient for either problem

The natural first algorithm for both problems is the same: compute pairwise cosine similarity over text embeddings, threshold, group. This is wrong for the deliberation setting, for the same underlying reason in both cases. Modern text embeddings encode **lexical and topical proximity** — shared subjects, shared vocabulary, shared sentence structure. They do **not** encode propositional content reliably:

- **Same-topic, opposite-stance.** “*Raise taxes on wealth above ten million*” vs. “*Lower taxes on wealth above ten million.*” Near-identical token distributions and syntax; cosine in `text-embedding-3-small` typically above 0.92; semantic opposites.
- **Same-topic, different recommendation.** “*Prioritize economic growth over environmental protection*” vs. its reverse. Not strict opposites, but not duplicates either.
- **Same-frame, different magnitude.** “*Add one lane to highway 5*” vs. “*Add three lanes to highway 5.*” Different proposals about the same project, ranked as nearly identical.

These are common in deliberative datasets. The cost of a false-positive merge is high: it conflates positions, hides minority views, and corrupts the evaluation signal — the very thing the system was meant to repair. The unifying principle of the architecture is therefore to treat **embedding cosine similarity as a candidate generator, not a decision rule**. The decision is delegated either to community evaluation data (in clustering) or to a separate semantic-equivalence judgment (in synthesis).

1.3 The architecture in one paragraph

A single embedding space (OpenAI `text-embedding-3-small`, 1 536-d) underlies the system. **Idea synthesis** runs as two coexisting pipelines on the same option pool. A *live single-option pipeline* fires on every option write: it ANN-searches the option against existing clusters, attaches above tuned thresholds (with a cohesion “snowball brake”), and otherwise asks an LLM whether the top candidate pair can be merged into a synthesized proposal (a *synth*) or only labelled as a shared theme (a *topic-cluster*). A *bulk admin pipeline* forms candidate clusters by **ANN-cosine threshold edges plus connected components**, then verifies each candidate cluster with a **medoid-anchored two-tier cosine-banded LLM judge** before generating per-group proposals or topic labels. Both pipelines write through a single integration primitive (`performIntegration`) and inherit a per-user-deduplicated evaluation aggregation. A separate, fully-deployed **topic-clustering** pipeline produces a taxonomy-driven framing (LLM taxonomy → normalization → per-category UMAP→DBSCAN → naming). A **hybrid text-rating clustering** design — blending text embeddings with an 8-dimensional evaluation vector — exists as implemented primitives but is *not wired*

into a running pipeline (§4). A set of scheduled consolidation jobs repairs the live pipeline’s residual fragmentation, and an operational-hygiene layer (run identity, mandatory tagging, reversible-by-construction membership, idempotent clean-then-rebuild) makes synthesis safe to re-run.

Deployment status, stated up front. The synthesis subsystem is gated behind environment feature flags that **all default OFF** (`SYNTHESIS_LIVE_SYNTH_ENABLED`, `SYNTHESIS_BULK_CLUSTER`, `SYNTHESIS_TWO_TIER_JUDGE`, `SYNTHESIS_ASYNC_JOB_MODE`, `SYNTHESIS_CLUSTER_AWARE_POLARIZATION`, `SYNTHESIS_BAYESIAN_PREFILTER`), with an `EMERGENCY_DISABLE_SYNTHESIS_FLAGS` panic switch that forces every flag off. Topic clustering and `performIntegration`-based manual merges run without these flags. Hybrid clustering runs nothing.

2. Embedding Infrastructure

2.1 Choice of embedding model

We use OpenAI `text-embedding-3-small`, producing 1 536-dimensional vectors, chosen for: multilingual coverage (Freedi serves English, Hebrew, Arabic, Spanish, German, and Dutch concurrently); cost efficiency (a backfill of 10 000 options costs on the order of \$0.02); and established cosine behaviour at scale. An earlier architecture experimented with Gemini `text-embedding-004` (768-d); the current system standardizes on the OpenAI space and uses Gemini for LLM-as-judge text generation only.

2.2 Context-aware generation

Embeddings are generated with the parent question included as context:

$$e_{\text{text}}(p_i) = \text{Embed}(\text{"Question: } Q \setminus n \text{ Answer: } t_i\text{"}) \in \mathbb{R}^{1536},$$

producing representations *relative to the deliberative context* rather than in absolute semantic space.

2.3 Storage and retrieval

Each option’s embedding is cached on the option document (`embedding` field) and indexed by Firestore’s native flat vector index, parameterized by (`parentId`, `embedding`, `dimension=1536`). Approximate-nearest-neighbour queries via `findNearest` (distance measure `COSINE`) return top- k neighbours within a parent in roughly 50–100 ms; the vector-search service over-fetches `limit` $\times$ 3 and filters hidden/threshold in memory. Embeddings are generated asynchronously after option creation. Both pipelines pre-flight an embedding-coverage check and offer a backfill action if coverage is insufficient.

3. Idea Synthesis

This section addresses the *fragmentation* problem of §1.1: collapsing paraphrases of one idea into a single canonical proposal whose evaluation is the per-user-deduplicated aggregate of its members’.

The same machinery produces *topic-clusters* — groupings of distinct-but-related ideas under a shared theme label — when an LLM judges that members share a subject but cannot honestly be merged.

3.1 Notation and cluster kinds

Let Q be a question with option set $S = \{s_1, \dots, s_N\}$. Each option s_i has text $\text{text}(s_i)$, a normalized embedding $\mathbf{v}_i \in \mathbb{R}^{1536}$, and an evaluation summary. Synthesis produces disjoint groups, each of one of two kinds:

Kind	Members	LLM artifact	derivedByPipeline
Synth	paraphrases of one idea	generated merged title + description	'synthesis'
Topic-cluster	distinct ideas on a shared theme	generated theme label only	'topic-cluster'

The choice is made at spawn time by `generateSynthesizedProposal`: if a coherent merged proposal can be written from the members, the result is a synth; if the model returns `cannotSynthesize`, the system falls back to `generateTopicLabel` and the result is a topic-cluster. Both shapes use the same **Statement** schema, the same `integratedOptions` member list, and the same aggregation infrastructure (§6).

3.2 The live single-option pipeline

`runSinglePipeline` (`functions/src/synthesis/pipeline/runSinglePipeline.ts`) runs on every option write and on threshold-cross events, making one cluster decision at a time with bounded LLM cost. Cosine is a **candidacy gate**, never a merge rule — the LLM is the synth-vs-topic-cluster judge.

Pre-checks. The pipeline skips work if the option is already a live member of a cluster, if the trigger is a continuous source and continuous synthesis is disabled, or if the option’s evaluator count / consensus falls below configured minimums (bypassed for admin-initiated runs). Crucially, membership is checked *authoritatively* via `findClustersContainingMember` (an `integratedOptions array-contains` query), because a member option does not itself carry `integratedOptions` — that field lives on the cluster document. This closes a cross-cluster double-claim bug (§9.2).

ANN search + evidence expansion. The pipeline searches the option’s parent with `findSimilarByEmbedding(embedding, parentId, { limit: NEIGHBOR_LIMIT = 10, threshold: reviewLowerBound })`. For each candidate cluster it computes a *best-evidence* score

$$\text{bestEvidence}(C) = \max \left(\sigma(\mathbf{v}, \text{title}(C)), \max_{m \in C \cap \text{neighbors}} \sigma(\mathbf{v}, m) \right),$$

then a second stage (`expandClusterEvidenceViaFullMembers`) batch-fetches *every* member’s embedding and promotes `bestEvidence` to the **average of the top-2 member cosines** (requiring ≥ 2 members, so a single outlier cannot drag a cluster over the gate). The transitive-via-member term is load-bearing: LLM-merged synth titles are abstracted and shortened, so the cluster *title*’s cosine to a long paraphrase often drops well below the cosine of the original *members*. Without this bump, a fresh paraphrase would spawn a duplicate synth sharing a member with the original.

The five passes.

Pass	Action	Condition	LLM cost
1 — Synth attach	attach to highest-evidence synth, subject to a cohesion gate	<code>bestEvidence</code> $\geq$ <code>attachThreshold</code> and newcomer fits centroid or member quorum	0
2 — Topic-cluster attach	attach to highest-evidence topic-cluster	<code>bestEvidence</code> $\geq$ <code>clusterThreshold</code> (lenient, no cohesion gate)	0
3 — Spawn (band-routed)	route the top non-member candidate pair by cosine band	top plain candidate $\geq$ <code>clusterThreshold</code>	1 LLM (synth) or 2 (synth + topic-label fallback on <code>cannotSynthesize</code>)
4 — Review	queue pair for admin review	top candidate in <code>[reviewLowerBound, clusterThreshold)</code>	0
5 — Singleton	leave option as-is (no cluster doc written)	no candidate above <code>reviewLowerBound</code>	0

Two refinements distinguish the deployed pipeline from a naive five-pass tree:

- **Cohesion gate (the “snowball brake”).** Pass 1 no longer attaches on a single close member. The newcomer must also fit the cluster centroid or a member quorum (`SYNTH_COHESION_QUORUM` = 0.5), preventing a synth from snowballing outward along a chain of pairwise-close but collectively-incoherent members.
- **Band-routed Pass 3.** Spawning is routed by `routeByCosine` over four bands defined by the thresholds below. A pair in `[clusterThreshold, synthLowerBound)` spawns a topic-cluster *directly* (no wasted synth attempt); a pair $\geq$ `synthLowerBound` attempts a synth and falls back to a topic-cluster on `cannotSynthesize`.

Default thresholds (`DEFAULT_SYNTHESIS_SETTINGS`, `pipeline/types.ts`; per-question overrides resolve from `statementSettings.synthesis`):

Setting	Default	Role
<code>attachThreshold</code>	0.85	synth-attach gate (Pass 1)
<code>synthLowerBound</code>	0.78	synth-vs-topic spawn band split (Pass 3)
<code>clusterThreshold</code>	0.60	topic-cluster attach/spawn gate (Pass 2/3)
<code>reviewLowerBound</code>	0.45	ANN search floor + review-queue gate (Pass 4)
<code>enabled</code>	<code>false</code> (<code>true</code> for Mass-Consensus questions)	live-synth master switch

Dedup on spawn. `spawnClusterFromPair` first runs `pairAlreadyClustered` (an `integratedOptions` array-contains query for each member) and returns without creating a duplicate if any visible cluster already contains either member. On success it stamps `derivedByPipeline`, `synthesisMechanism: 'live-spawn'`, `liveSynthOrigin: 'spawn'`, enqueues a cluster recompute, and writes a `_synthBulkRequests/{parentId}` marker so the scheduled re-truth sweep (§3.5) later reconciles structure.

3.3 The bulk admin pipeline

The bulk pipeline is invoked from the synthesis admin panel and from scheduled sweeps. **The candidate generator is ANN-cosine threshold edges plus connected components — not UMAP→DBSCAN.** This is the single most important correction relative to the earlier draft, and it is grounded in a production incident (§7).

Candidate geometry (`buildCandidateClusters`, `candidateClusters.ts`).

1. `buildCandidateEdges` runs an ANN cosine search per candidate option (top- K , canonical de-duplicated edges, bounded concurrency) and keeps every edge with cosine $\geq \tau$.
2. `UnionFind` computes connected components over those edges.
3. Components of size ≥ 2 become candidate clusters; every option with no $\geq \tau$ neighbour is left a singleton.

The threshold defaults to `SYNTHESIS_CANDIDATE_THRESHOLD = 0.92` (`resolveCandidateThreshold`). This geometry **preserves singletons** — the ~90 % of a real corpus that are distinct ideas are left alone rather than forced into buckets — which is exactly the property UMAP→DBSCAN lacks (§7). `bulkClusterByEmbedding` (the former UMAP→DBSCAN candidate generator) is no longer wired into any synth path; only its `cosineSimilarity` / `meanVector` helpers survive as utilities.

Two-tier judge (`twoTierJudge.ts`). For each candidate cluster C , the pipeline verifies membership with a medoid-anchored scheme rather than $O(|C|^2)$ all-pairs LLM verdicts:

1. **Medoid.** Pick the member with the highest mean cosine to the rest (linear, no LLM).
2. **Band each non-medoid member** m by $\sigma(m, \text{medoid})$:

Band	Condition	Action
Auto-accept	$\sigma \geq 0.94$	equivalent to medoid, no LLM
Gray	$0.60 \leq \sigma < 0.94$	one cached LLM verdict, member $\leftrightarrow$ medoid
Auto-reject	$\sigma < 0.60$	dissent, no LLM

3. **Tally.** With agreement fraction ≥ 0.80 , keep the agreed subset (dissenters are outlier noise, telemetry only). With $0.50 \leq \text{agree} < 0.80$, keep the agreed subset **and** route the dissent subset through `refineComponent` (complete-linkage) to recover any internal sub-clique as its own verified cluster. Below 0.50, drop the cluster — cosine alone was not enough signal. In *both* keep tiers the output cluster contains only the medoid plus auto-accepted plus LLM-agreed members; auto-rejected members are never in the output.

A hard run-level cap `maxLlmCalls = min(2000, |workingSet| × 0.2)` prevents budget blowouts; over-cap gray pairs fall back to cosine-only, tagged `verifiedBy: 'cosine-only'`, `llmCallsCapped: true`, and are surfaced in the admin UI for manual review.

The auto-reject band is 0.60, not the 0.82 an earlier draft reported. The code was deliberately lowered because same-idea and different-idea cosine distributions overlap: a high reject band silently demoted valid members to dissent. The gray band is therefore 0.60–0.94.

Async-job phases (`asyncJob/phases.ts`). When run as a resumable Cloud-Function chain, the job progresses through `loading` → `clustering` → `verifying` → `proposing` → `ready-for-review`. “Cleaning” is folded into `loading` (it calls `dissolveQuestionSynthesis` for an idempotent clean-then-rebuild, §6.3); there is no separate committing phase — the job halts at `ready-for-review`, and an admin confirms/commits through a separate callable that invokes `performIntegration`.

Feature-flag gating. The synchronous `synthesizeIdeasPreview` / `synthesizeIdeasExecute` callables take the ANN-cosine bulk path only when `shouldUseBulkSynthesisPath()` — i.e. `SYNTHESIS_BULK_CLUSTER` and `SYNTHESIS_TWO_TIER_JUDGE` both ON (both default OFF). With the flags off they fall back to a legacy path (`buildCandidateEdges` → per-pair LLM equivalence → `UnionFind` → `refineComponent`). Neither path uses UMAP→DBSCAN. The scheduled sweeps and the async-job clustering phase call `buildCandidateClusters` unconditionally.

3.4 The four-way verdict and verdict cache

Both pipelines and the complete-linkage refinement share one LLM prompt and one persistent verdict cache. The judge returns one of four verdicts, broadening a binary opposite/not-opposite check:

Verdict	Interpretation	Action
same	paraphrase / near-duplicate of the same proposal	keep edge
related	same topic, different stance / recommendation / magnitude	drop edge
different	embeddings coincidentally close; unrelated	drop edge
opposite	explicit contradiction on the same subject	drop edge

The prompt explicitly separates paraphrase (**same**) from any form of disagreement, forcing a deliberate choice rather than a permissive “yes-similar.” Each Gemini 2.5-Flash call batches up to 20 pairs (~1–3 s, on the order of 4×10^{-4} USD); above 20 the output JSON becomes unreliable.

Verdicts are persisted in a `synthesisVerdicts` collection keyed by a content-addressable, order-symmetric hash

$$\text{pairKey} = \text{sha1}(\min(h_A, h_B) \parallel \text{"|"} \parallel \max(h_A, h_B)), \quad h_X = \text{sha1}(\text{normalize}(\text{text}(X))),$$

with (textHashA, textHashB, verdict, reason, modelId, promptVer, createdAt). A row is a hit only if modelId and promptVer match the running config *and* both text hashes match; bumping promptVer invalidates the whole cache without a migration. Fallback verdicts from an LLM failure are **not** cached, so a transient error never freezes a permanent decision. The dominant practical effect is on re-runs: only edges touching new or edited options pay the LLM cost, making incremental synthesis cheap by construction.

3.5 Scheduled consolidation and admin entry points

The live stage is deliberately bounded and leaves residual fragmentation at burst arrival; a set of scheduled jobs converges the structure. All confirmed in `index.ts`:

Function	Schedule	Role
<code>processSynthesisQueue</code>	every 1 min	drains <code>synthesisQueue/{questionId}/items</code> ; each <code>process-option</code> item replays <code>runSinglePipeline</code>
<code>fn_synthesisBulkFlush</code>	every 2 min	reads <code>_synthBulkRequests</code> ; ANN-cosine buildCandidateClusters attach-only reconciliation (skips parents active in last 30 s)
<code>fn_synthesisReJudge</code>	every 10 min	cross-synth merge: merges donor→recipient when cross-member top-2-avg cosine ≥ <code>REJUDGE_MERGE_THRESHOLD</code> = 0.82
<code>fn_clusterRecomputeFlush</code>	every 1 min	drains <code>_clusterRecomputeQueue</code> , recomputes cluster evaluation + polarization (gated by <code>clusterAwarePolarization</code> , default OFF)

The admin “Synthesize” button is a live-pipeline replay, not the bulk pass. `synthesizeNow` enqueues one `process-option` item per eligible option into `synthesisQueue/{questionId}/items`; `processSynthesisQueue` drains them through `runSinglePipeline`. The bulk ANN-cosine pass is reached only through `synthesizeIdeasPreview` / `synthesizeIdeasExecute` and the async job. This distinction matters for anyone reproducing the pipeline: the Firebase emulator executes Firestore triggers but does **not** fire `onSchedule` functions, so under the emulator the `synthesizeNow` queue never drains and only the live stage runs (§8.2).

3.6 Member hiding happens only on the execute/integration path

`performIntegration` — which creates the cluster statement, migrates evaluations, and sets `hide: true`, `integratedInto` on the originals (§6.1) — is called from `synthesizeIdeasExecute` (bulk-

confirm), `fn_integrateSimilarStatements` (manual merge), and `fn_globalCluster`. Neither the live pipeline nor the `synthesizeNow` queue replay calls it; both only append to a cluster document’s `integratedOptions`. So a question whose clusters were built only by the live pipeline shows all source options unhidden — the collapsed view is a property of the execute step, not of cluster membership. The cross-synth reJudge merge hides the *donor synth*, not raw members.

4. Hybrid Text–Rating Clustering (designed; primitives implemented; not wired)

This section documents a design whose mathematical primitives are implemented in the codebase but which **does not run as a pipeline**. We keep it because the primitives are real, the design is sound, and the honest status is itself the point: an earlier draft presented hybrid clustering as a deployed pillar, which the source code does not support.

4.1 The design

For each proposal p_i with n_i evaluators, form a composite vector blending the text embedding $\mathbf{e}_i \in \mathbb{R}^{1536}$ with an 8-dimensional rating vector $\mathbf{r}_i$, weighted by an adaptive parameter $\alpha(n_i)$ and a square-root dimensionality correction so each component contributes its intended weight to cosine:

$$\mathbf{v}_i = [\alpha(n_i)\sqrt{D_{\text{text}}/D_{\text{total}}}\mathbf{e}_i, (1 - \alpha(n_i))\sqrt{D_{\text{rating}}/D_{\text{total}}}\mathbf{r}_i] \in \mathbb{R}^{1544}.$$

The adaptive weight (`computeAlpha`, `hybrid-vector-service.ts`) is

$$\alpha(n_i) = \frac{1}{1 + \beta \log_2(1 + n_i)}, \quad \beta = 0.3,$$

transitioning from text-only at $n_i = 0$ (crossover $\alpha = 0.5$ near $n \approx 37$) to rating-dominated as evaluations accumulate — the design move that would let clustering *learn stance from votes* without stance being encoded in the text. The 8 rating dimensions (all derived from pre-computed aggregation fields, no extra queries) are: mean evaluation; Bayesian-smoothed WizCol consensus; confidence-adjusted agreement; like- mindedness; pro-evaluator ratio; density; polarization (1–like-mindedness); and intensity. K-means with k-means++ initialization and an elbow-method k selection ($k_{\min} = 3$, $k_{\max} = \min(20, \lfloor n/2 \rfloor)$), and a post-clustering Gemini negation check on within-cluster pairs above cosine 0.85, are also implemented as functions (`kmeans-service.ts`, `negation-detection-service.ts`).

4.2 What actually runs

Only two fragments of this design execute in production:

- **Staleness marking.** Evaluation triggers call `markHybridEmbeddingStale`, setting `hybridEmbeddingStale: true` on the affected proposal.
- **A zero-rating initial vector.** On option creation, if `enableHybridClustering` is set, a hybrid vector is written **with an all-zero rating half and 0 evaluators** — i.e. text-only.

Everything else is orphaned: `hybridEmbeddingStale` is written but **never queried**; there is **no 15-minute scheduled recompute**; `kmeans()` / `selectOptimalK()` / `negation-detection-service` are **never called** from any production path; and **no hybrid-auto framing is ever produced**. Wiring the scheduled recompute (staleness sweep → recompute hybrid vectors with real ratings → k-means → negation split → write hybrid-auto framing) is the outstanding work to make this a live feature. Until then, hybrid clustering should be described as *scaffolding*, not a deployed capability.

5. Topic Clustering (deployed)

A separate, fully-deployed pipeline (`functions/src/services/topic-cluster/`, admin callable `triggerTopicClusterPipeline`) produces a deliberative *taxonomy* framing independent of evaluations:

1. **Taxonomy derivation (LLM)**. A category list (up to `TAXONOMY_MAX_CATEGORIES = 8`) is generated for the question and cached.
2. **Normalization (LLM)**. Each option is mapped to canonical actions and assigned to one or more categories, cached.
3. **Per-category clustering**. Embed canonical actions; project 1 536-d cosine space to `UMAP_TARGET_COMPONENTS = 10`-d Euclidean via UMAP with a deterministic seed 42; apply DBSCAN with `DBSCAN_EPS = 1.0`, `DBSCAN_MIN_SAMPLES = 3`. Buckets below `UMAP_MIN_ITEMS = 10` use a union-find fallback at similarity 0.7.
4. **Noise recovery**. DBSCAN noise points reassign to the nearest centroid (cosine in *original* embedding space) above `NEAREST_CENTROID_THRESHOLD = 0.6`, else “uncategorized.”
5. **Cluster naming (LLM)**. Each cluster is named from its members; a framing is written.

UMAP→DBSCAN is appropriate *here* — coarse thematic blobs are acceptable for a taxonomy — which is precisely why it was demoted from the synth path, where singleton preservation is essential (§7). Topic clustering and synthesis produce alternative framings a facilitator can compare side-by-side.

6. Shared Infrastructure

6.1 The merge primitive — `performIntegration`

Manual merges and bulk-synthesis confirmations call the same primitive (`functions/src/integrate/performIntegration`). Effects:

1. Create a `Statement` with `isCluster: true`, `integratedOptions: [memberIds]`, `derivedByPipeline: 'synthesis' | 'topic-cluster'`, and (per-run) `synthesisRunId`, `synthesisMechanism`.
2. Migrate member evaluations by per-user deduplication (§6.2).
3. Hide originals: `hide: true`, `integratedInto: <newId>`.
4. Bump the parent’s `lastChildUpdate` so client subscriptions invalidate.

A hybrid cluster, a topic cluster, and a synthesized solution therefore share one data shape — they *are* the same shape, reached by different algorithms. It is reversed by `reverseIntegration`.

6.2 Per-user-deduplicated evaluation aggregation

When a cluster is built, member evaluations are not summed naively — a user who voted on three variants of one idea must count once. For each user u who evaluated any member of group G ,

$$\hat{e}_{u,G} = \frac{1}{|G \cap \text{evaluated}(u)|} \sum_{s \in G \cap \text{evaluated}(u)} e_{u,s},$$

and the group evaluation is the mean of $\hat{e}_{u,G}$ over unique evaluators; the cluster’s evaluator count is the number of unique users, **not** the sum of per-member counts. Without this, both layers would amplify the engagement of any user who voted on multiple variants — the exact distortion the system exists to remove.

Two implementations realize the principle: `migrateEvaluationsToNewStatement` (called by `performIntegration`) *materializes* new per-user evaluation documents on the target and excludes neutral votes; while `computeClusterEvaluationFromRawEvals` (the live-recompute path) reads member evaluations on the fly, includes neutral votes, and adds a **direct-vote-wins** refinement — if a user voted on the cluster statement itself, that direct vote overrides their member-vote average. Both group by `evaluatorId`, so the unique-evaluator invariant holds either way. The aggregate is a full `StatementEvaluation` (mean sentiment, Bayesian-smoothed consensus, agreement index, like-mindedness, confidence index, pro/con counts), so downstream UI reads a cluster’s stats exactly as a plain option’s.

6.3 Keeping the live cluster aggregate current, and clean-then-rebuild

The aggregated evaluation is **denormalized onto the cluster document**, kept fresh by a queue-and-flush mechanism: `enqueueClusterRecompute(clusterId)` writes a marker on any membership or member-text change (and, when `clusterAwarePolarization` is on, on member-evaluation change via `findClustersContainingMember`); `fn_clusterRecomputeFlush` (every 1 min) drains the queue and recomputes. Bursts collapse to a single queued marker per cluster.

For idempotent re-runs, `dissolveQuestionSynthesis` (`derivedDocs.ts`) reverses proper clusters via `reverseIntegration`, hard-deletes malformed/legacy derived docs, re-shows orphaned hidden options, and deletes stale evaluations — driven by an **inclusive** classifier `isDerived(doc) = nonEmpty(integratedOptions) ∨ isCluster ∨ derivedByPipeline ∨ liveSynthOrigin`, which catches legacy untagged outputs that a `derivedByPipeline`-only cleanup would miss. Manual, creator-locked clusters (`isCluster ∧ titleLockedByCreator`) are preserved.

7. Production Hardening and the UMAP→DBSCAN Correction

The bulk pipeline’s candidate geometry was changed in response to a concrete production failure, which also motivated the operational-hygiene layer.

7.1 The incident

Running the bulk path on a real question (`hwEoIYX2tYHJ`, `wizcol-app`, 252 Hebrew options) produced visible chaos: giant over-merged buckets, 1-member “synths”, empty topic headers, duplicate synths, and 26 user options hidden with no recovery link. Diagnosis found two independent causes.

Wrong candidate geometry. UMAP projects 1 536-d embeddings onto a connected low-dimensional manifold, so DBSCAN returns *a few big blobs with zero singletons at every ε* (measured: $\varepsilon = 1.0 \rightarrow 1$ cluster; $0.45 \rightarrow 7$ clusters, 0 noise; $0.1 \rightarrow$ clusters of 133/60/59). Real deliberation is the opposite shape — mostly distinct ideas with a few paraphrase clusters. Raw-cosine connected components on the same 252 options recover the true shape:

cosine threshold	options in clusters	singletons	note
≥ 0.95	9	243	very confident dupes only
$\geq \mathbf{0.92}$	22 (5 clusters)	230	the live attach gate — clean
≥ 0.90	47	205	a 29-member chained blob appears
≥ 0.88	101	151	74-member blob
≥ 0.85	180	72	172-member runaway chain

Feeding the cosine- ≥ 0.92 candidates to the existing `twoTierJudge` returned **3 genuine synths + 245 standalone** (15 LLM calls, ~22 s); the judge correctly dropped a 13-member blob of short generic texts and split “inside” vs. “outside polling stations.” **The judge was correct; the candidate generator was the defect.** The fix was to generate candidates the way the live path already did — ANN cosine + connected components — which is now the deployed bulk geometry (§3.3). UMAP→DBSCAN is retained only for topic-level grouping (§5), where coarse blobs are acceptable.

7.2 Operational hygiene

A second class of failure was operational, not algorithmic: outputs from multiple historical runs coexisted with no run identity or cleanup discipline (95 leftover derived docs, 78 of them carrying no `derivedByPipeline` field and thus indistinguishable from real options), 26 options hidden without `integratedInto` (orphaned when their cluster was later deleted), and stale membership causing double-counting. The hardening response — now landed — makes synthesis identifiable, reversible, cleanable, and safe to re-run:

- **Run identity + mandatory tagging.** Every derived doc carries `derivedByPipeline`, `synthesisRunId` (per-run UUID), and `synthesisMechanism` $\in \{\text{'bulk'}, \text{'live-spawn'}, \text{'live-attach'}\}$, stamped at `performIntegration` and `spawnClusterFromPair`.
- **Inclusive cleanup.** `isDerived` (§6.3) catches legacy untagged outputs; `dissolveQuestionSynthesis` runs in the loading phase so a re-run is self-cleaning.
- **Reversible-by-construction membership.** An option is hidden only together with a valid `integratedInto`; dissolving a cluster un-hides its members in the same batch. No orphans by construction.

- **Dedup.** `spawnClusterFromPair` skips when a visible cluster already contains either member.
- **Parameter discipline.** No single default ε /threshold is shipped blindly; the chosen τ and judge bands are recorded per run, and cross-language portability of $\tau = 0.92$ is validated by a shadow sweep rather than assumed.
- **Shadow mode.** A no-write path writes proposed groupings to a review collection (mirroring `_liveSynthCandidates/`) for human audit on real data before promotion.

8. Verification, Benchmarks, and Empirical Reproduction

8.1 Testing surface

Each pure step is unit-tested with no Firestore I/O: `buildCandidateClusters` (candidate geometry, singleton preservation), `twoTierJudge` (auto-accept / gray / auto-reject branches, keep/split/drop tally, LLM cap, dissent refinement), `refineComponent` (complete-linkage chaining mitigation), and `runSinglePipeline` (the live decision tree). Emulator scripts (`runTopicClusterEmulator.ts`, `inspectSynthBenchmark.ts`) close the loop against a local Firestore.

8.2 The benchmark harness and its lesson

The canonical seed (`scripts/seedSynthBenchmark.data.json`) encodes one question with a deterministic structure — **2 topics $\times$ 2 synths per topic $\times$ 10 paraphrases = 40 options** — the smallest corpus that exercises every live decision (synth attach, topic-cluster attach, synth spawn, topic-cluster spawn, singleton). A correct run yields 2 topic-clusters, 4 synths (~10 members each), and 40 source options with absorbed ones hidden.

Reproduction surfaced two structural facts. First, **the scheduled consolidation layer does not run under the emulator** (the emulator fires Firestore triggers but not `onSchedule` cron), so the seed-and-inspect loop exercises only the live stage; the `synthesizeNow` queue never drains. Second, **live-only output is fragmented by design**: on a representative run the live stage produced headline-correct counts (2 topic-clusters, 4 synths) over a defective interior — one synth fragmented, one never formed, a topic-cluster spawned at cosine 0.604 mixing unrelated paraphrases. The cross-synth reJudge merge (§3.5) cleanly repaired the duplicate once invoked (cross-member cosine 0.851 $\geq$ the 0.82 merge gate), confirming that the *scheduled* layer, not the live stage, is what converges the structure. The lesson: benchmark success criteria must test **disjointness and per-synth purity**, not counts alone — counts can match by coincidence while the grouping is wrong.

By contrast, running the ANN-cosine bulk geometry over the same 40 paraphrases separated the four groups exactly (four clusters of ten, 100 % pure against ground truth, zero noise; within-group cosine 0.86–0.95, cross-group well below), a direct confirmation of the singleton-preserving design of §3.3 and §7.1.

8.3 Recommended regression surface

Run the benchmark on any change to: the embedding model or context format (§2); the live thresholds, cohesion gate, or best-evidence index (§3.2); the bulk candidate threshold τ (§3.3); the two-tier judge bands (§3.3); or the four-way verdict prompt (§3.4). Each is the kind of change that passes every unit test yet silently degrades end-to-end behaviour.

9. Scaling Analysis

9.1 Live pipeline (per option write)

Bounded at ≤ 2 LLM calls per write regardless of N . Cost is dominated by the ANN search (~ 50 – 100 ms) and, when it fires, the spawn LLM call (~ 1 – 3 s). The verdict cache short-circuits the spawn call on any re-write that does not change the option’s text. Full-member evidence expansion costs $O(\sum_{C \in \text{candidates}} |C|)$ Firestore reads, bounded by neighborhood size.

9.2 Bulk pipeline (per admin run)

Let N' be the pre-filtered option count, K the cluster count, g the gray-band fraction (empirically 0.2–0.4 of non-medoid members).

Phase	Complexity	Bottleneck
Coverage + pre-filter	$O(N)$ Firestore reads	linear scan
Candidate geometry (ANN edges + UnionFind)	$O(N')$ ANN queries, bounded concurrency	Firestore vector search
Two-tier judge — cosine scoring	$O(\sum_C C)$ in-memory	medoid pick is linear per cluster
Two-tier judge — gray-band LLM	$O(\lceil g \sum_C (C - 1) / 20 \rceil)$ batched calls, capped at <code>maxLlmCalls</code>	network; cached on re-runs
Dissent-subset complete-linkage	$O(K_{\text{split}} \cdot \overline{ D }^2 / 20)$ extra calls	only for 50–80 % clusters
Per-cluster proposal generation	$O(K)$ LLM calls	one per verified cluster
<code>performIntegration</code> + <code>finalize</code>	$O(K)$ writes + $O(K)$ recomputes	linear

LLM cost scales with the **gray-band member count**, not candidate-edge count; auto-accept and auto-reject members consume zero budget. Indicative wall-clock: small ($N \leq 500$) ~ 30 s cold / ~ 5 s warm; medium ($N \leq 10\,000$) ~ 2 min cold / ~ 15 s warm. For $N = 10\,000$ cold: embedding backfill $\sim \$0.02$, gray-band judging $\sim \$0.20$, proposal generation $\sim \$0.05$ — total marginal cost per run well under one dollar, cents on a warm cache. Re-runs are cheap by construction because only edges touching changed options pay the LLM cost.

9.3 The membership-idempotence fix

`runSinglePipeline`’s original pre-check tested `option.integratedOptions.length > 0` — but membership lives on the `cluster` doc, so a clustered option still carries an empty `integratedOptions` and was re-processed as fresh, letting re-runs add it to a second cluster (observed: re-draining a 40-option queue raised multiply-claimed options from 1 to 8). The fix queries `findClustersContainingMember` and skips when any live, non-hidden cluster owns the

option, so a “Synthesize” re-run or queue replay is now idempotent with respect to membership. The bulk path remains the right tool for *re-clustering* (each option lands in exactly one component).

10. Limitations and Open Questions

1. **Hybrid clustering is unwired (§4).** The primitives exist; the scheduled recompute, k-means invocation, negation split, and **hybrid-auto** framing do not. Until wired, evaluation-aware clustering is not a running capability, and the cold-start argument for learning stance from votes is theoretical.
 2. **LLM judge as a single point of trust.** The four-way verdict is the merge decision rule for the pairs it sees; auto-accept ($\sigma \geq 0.94$) and auto-reject ($\sigma < 0.60$) bypass it entirely on cosine alone. Both bands were calibrated pragmatically, not from a deliberation-specific study. Mitigations: a held-out hand-labelled calibration set with a reported four-way confusion matrix on each model/version change; cosine-only telemetry surfaced for manual review when the cap is hit; human-in-loop preview.
 3. **Frame-vs-magnitude pairs** (“add one lane” / “add three lanes”) are the most prompt-sensitive boundary; the prompt favours **related**, which we believe correct but recommend calibrating before deployment.
 4. **Threshold portability.** $\tau = 0.92$ and the judge bands were tuned on largely English/Hebrew corpora; cross-language portability should be validated by shadow sweep, not assumed.
 5. **Evaluation drift.** A cluster’s denormalized aggregate refreshes on membership change and (only when **clusterAwarePolarization** is on) on member-evaluation change. With the flag off — the default — aggregates drift between membership changes. Turning it on by default, once validated on high-traffic questions, closes the gap.
 6. **Cluster stability.** Frequent re-clustering can migrate proposals between clusters, confusing users who track one over time; a stability-weighted assignment is a natural extension.
 7. **Reversibility UX.** **performIntegration** is reversible in data (**reverseIntegration**) but the platform exposes no admin “unsynthesize” action; this should precede broad deployment.
 8. **Cross-question synthesis is out of scope** by design — each question is its own deliberative unit with its own evaluator pool.
 9. **Substrate-dependent convergence.** Correctness is *eventual* and depends on the scheduled sweeps; any environment without a cron scheduler (notably the emulator) leaves the live stage’s rough output as the terminal state (§8.2). Exposing the consolidation logic as on-demand callables would remove the dependency.
-

11. Conclusion

Three observations support the design. First, **two computational problems coexist** — fragmentation of paraphrases (synthesis) and organization of distinct ideas (clustering) — and they require different remedies even though both appear solvable by the same embedding-similarity tool. Second, **embeddings encode topical proximity, not propositional content**, so both layers treat similarity as a *candidate generator* and delegate the decision — to a medoid-anchored cosine-banded LLM judge in the bulk pipeline, to a per-spawn LLM judgment in the live pipeline, and (by design, not yet deployed) to community evaluation data in hybrid clustering. Third, **per-user-**

deduplicated evaluation aggregation is essential — it is the bridge between cluster structure and the consensus metrics participants actually see.

The system runs within standard cloud infrastructure — Firebase Cloud Functions, Firestore vector indexes, scheduled jobs — at marginal cost on the order of cents per run, scaling to tens of thousands of options per question. The deployed capabilities are: the live single-option synthesis pipeline, the bulk ANN-cosine + two-tier-judge pipeline, taxonomy-based topic clustering, per-user-deduplicated aggregation, and the operational-hygiene layer — all behind default-off feature flags for synthesis. Hybrid text-rating clustering remains implemented-but-unwired. We invite review of: the four-way verdict prompt and category boundaries; the auto-accept / auto-reject cosine bands and keep/split/drop thresholds; the cohesion gate and best-evidence index; the synth-vs-topic-cluster fork; the per-user aggregation; and the limitations of §10 — particularly the unwired hybrid layer, the LLM-judge single point of trust, and substrate-dependent convergence.

Appendix A — Reconciliation with the source code

This revision corrects an earlier draft (`clustering-and-synthesis-paper.md`, 2026-06-01) against the deployed implementation. The material corrections:

Earlier draft	Source code (verified 2026-07-09)
Bulk pipeline = in-memory UMAP→DBSCAN	ANN-cosine edges + UnionFind connected components (<code>buildCandidateClusters</code> , $\tau = 0.92$); UMAP→DBSCAN dead for synth, kept for topic clustering
Hybrid text-rating clustering is a deployed pillar with a 15-min scheduler	Primitives implemented but never called; no scheduler, no <code>hybrid-auto</code> framing; only staleness marking + a zero-rating initial vector run
Two-tier judge auto-reject < 0.82 Live <code>clusterThreshold</code> 0.65; flat synth-then-fallback Pass 3	Auto-reject < 0.60 ; gray band 0.60–0.94 <code>clusterThreshold</code> 0.60; <code>attachThreshold</code> 0.85; <code>synthLowerBound</code> 0.78; <code>reviewLowerBound</code> 0.45; band-routed Pass 3; Pass 1 cohesion gate
—	All synthesis feature flags default OFF; <code>EMERGENCY_DISABLE_SYNTHESIS_FLAGS</code> panic switch
Async phases: loading → cleaning → clustering → verifying → proposing → committing	<code>loading</code> (folds in cleaning) → <code>clustering</code> → <code>verifying</code> → <code>proposing</code> → <code>ready-for-review</code> ; commit is a separate admin callable
Topic clustering UMAP target 5-d	<code>UMAP_TARGET_COMPONENTS</code> = 10; <code>DBSCAN_EPS</code> = 1.0, <code>DBSCAN_MIN_SAMPLES</code> = 3, <code>NEAREST_CENTROID_THRESHOLD</code> = 0.6

Key source references: `functions/src/synthesis/pipeline/runSinglePipeline.ts`, `.../candidateClusters`, `.../twoTierJudge.ts`, `.../asyncJob/phases.ts`, `.../featureFlags.ts`, `.../derivedDocs.ts`,

.../liveSynth/clusterRecompute.ts, functions/src/fn_synthesizeIdeas.ts, functions/src/integrate/
functions/src/services/hybrid-vector-service.ts, functions/src/services/topic-cluster/
and the schema in packages/shared-types/src/models/statement/StatementTypes.ts.

References

- Arthur, D., & Vassilvitskii, S. (2007). K-means++: The Advantages of Careful Seeding. *Proc. SODA 2007*, 1027–1035.
- Ester, M., Kriegel, H.-P., Sander, J., & Xu, X. (1996). A Density-Based Algorithm for Discovering Clusters. *Proc. KDD 1996*, 226–231.
- Fishkin, J. S. (2018). *Democracy When the People Are Thinking*. Oxford University Press.
- McInnes, L., Healy, J., & Melville, J. (2018). UMAP: Uniform Manifold Approximation and Projection. *arXiv:1802.03426*.
- Niklaus, J., Chalkidis, I., & Stötzer, M. (2023). An Empirical Study on the Robustness of Transformer-based Embeddings to Negation. *Findings of ACL 2023*.
- Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. *Proc. EMNLP-IJCNLP 2019*, 3982–3992.
- Small, C., Bjorkegren, M., Erkkilä, T., Shaw, L., & Megill, C. (2021). Polis: Scaling Deliberation by Mapping High-Dimensional Opinion Spaces. *Recerca*, 26(1).
- Tessler, M. H., Bakker, M. A., Jarrett, D., et al. (2024). AI Can Help Humans Find Common Ground in Democratic Deliberation. *Science*, 386(6719).
- Freedi Project (2026). WizCol: Weighted Consensus Calculation for Collective Intelligence. *Freedi Technical Report*.